

SCS1308 - Foundations of Algorithms

Tutorial - 01

13.10.2026

What is an Algorithm and How to Analyse it?

```
for (int i = 0; i < n; i++) {
    ...
}
```

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        ...
    }
}
```

```
for (int i = 0; i < n; i++) {
    ...
}
for (int j = 0; j < m; j++) {
    ...
}
```

```
if (n > 1000) {
    for (int i = 0; i < n; i++) {
        ...
    }
} else {
    for (int j = 0; j < log(n); j++) {
        ...
    }
}
```

```
while (n > 1) {
    n = n / 2;
    ...
}
```

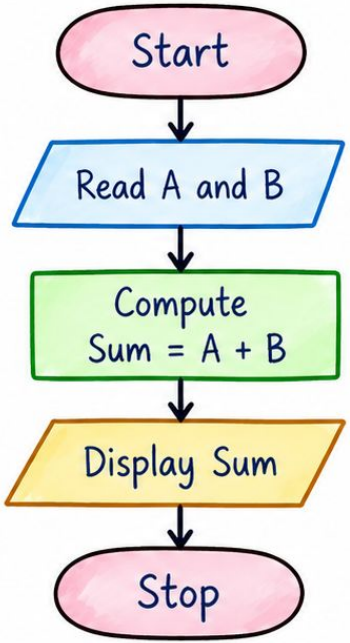
What is an Algorithm?

An **algorithm** is a **step-by-step procedure** to solve a problem or perform a task.

Key Characteristics

- Has clear, finite steps
- Takes input and produces output
- Each step is unambiguous
- Must end after a finite number of steps

Example : Adding two numbers



1. Start
2. Read A and B
3. Compute Sum = A + B
4. Display Sum
5. Stop

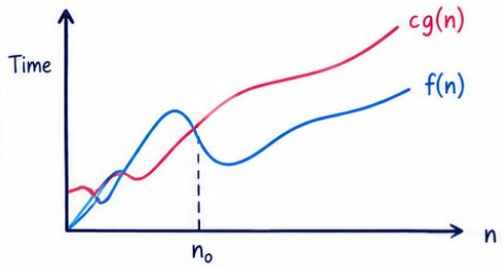
Algorithm Efficiency

- Efficiency depends on **time** and **space complexity.**
- Types of analysis :
  - Worst case (maximum time/space)
  - Best case (minimum time/space)
  - Average case (typical/expected time/space)
- Comparisons often focus on growth rates (Big-O, Omega, Theta).

What do they measure?

- **Time complexity** – how the running time grows with input size (n).
- **Space complexity** – how the memory usage grows with input size (n).

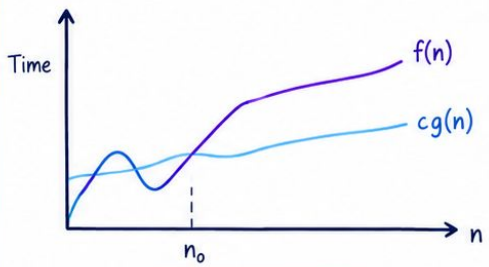
Big-O (Upper bound)



$f(n) = O(g(n))$

(f(n) is at most proportional to g(n))

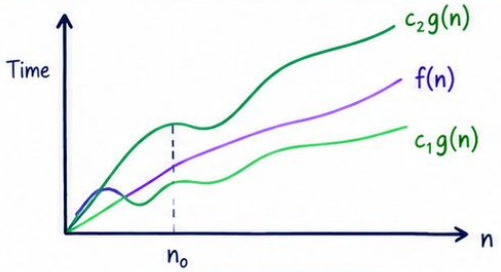
Omega (Lower bound)



$f(n) = \Omega(g(n))$

(f(n) is at least proportional to g(n))

Theta (Tight bound)



$f(n) = \Theta(g(n))$

(f(n) grows at the same rate as g(n))

Big-O, Omega and Theta Notation

These notations describe the **growth rate** of an algorithm's running time (or space) for large input sizes.

Big-O : $O(g(n))$

- Upper bound.
- Describes the worst-case growth rate.
- The running time is at most proportional to g(n) for large n.

$f(n) \leq c \cdot g(n)$ (for $n \geq n_0$)

Omega : $\Omega(g(n))$

- Lower bound.
- Describes the best-case growth rate.
- The running time is at least proportional to g(n) for large n.

$f(n) \geq c \cdot g(n)$ (for $n \geq n_0$)

Theta : $\Theta(g(n))$

- Tight bound.
- Describes both the upper and lower bound (same growth rate).
- The running time is proportional to g(n) for large n.

$c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)$ (for $n \geq n_0$)

Steps to analyse time complexity

1. Identify basic operations (e.g., comparisons, assignments).
2. Count how many times they execute as a function of input size n.
3. Write the running time T(n).
4. Keep only the dominant (highest growth) term and ignore constants and lower order terms.
5. Express the result using Big-O, Omega or Theta.

Example : Find the time complexity

```
Algorithm SumArray(A[0 ... n-1])
1   sum = 0
2   for i = 0 to n - 1
3   ⊦    sum = sum + A[i]
4   return sum
```

- Basic operation: sum = sum + A[i]
- Executed n times (once per iteration).
- Running time: $T(n) = c_1 + c_2 \cdot n$
- Ignore constants → $T(n) = \Theta(n)$

More examples

Algorithm	T(n)	Big-O	Omega	Theta
Access array element	c	$O(1)$	$\Omega(1)$	$\Theta(1)$
Linear search (worst case)	$c_1 n + c_2$	$O(n)$	$\Omega(1)$	$\Theta(n)$
Binary search	$c \log n$	$O(\log n)$	$\Omega(\log n)$	$\Theta(\log n)$
Nested loops (for i: 1..n, for j: 1..n)	$c n^2$	$O(n^2)$	$\Omega(n^2)$	$\Theta(n^2)$
Triple nested loops	$c n^3$	$O(n^3)$	$\Omega(n^3)$	$\Theta(n^3)$

Growth Rate Classes

We analyze functions asymptotically and group them based on their growth rate (how fast they grow as n becomes large).

- Constant : $\Theta(1)$ (e.g. $O(1)$)
- Linear : $\Theta(n)$
- Logarithmic : $\Theta(\log n)$
- n log n : $\Theta(n \log n)$
- Quadratic : $\Theta(n^2)$
- Polynomial : $\Theta(n^k)$
- Exponential : $\Theta(k^n)$ $(k > 1)$

→ Only the leading term is important.

→ Constants don't make a significant difference.

The following inequalities hold asymptotically :

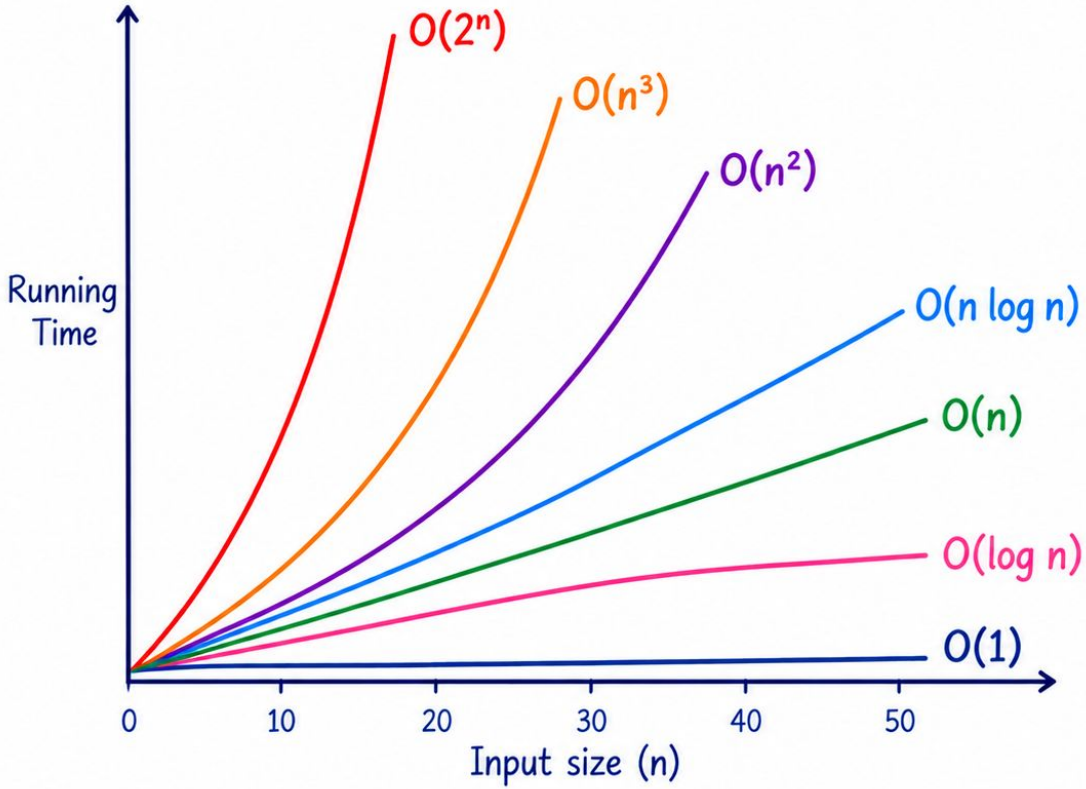
$$c < \log n < \log n^2 < n < n \log n < n^2 < n^3 < n^k < k^n < 2^n$$

n	$\log_2 n$	n	n^2	n^3	2^n
1	0	1	1	1	2
2	1	2	4	8	4
4	2	4	16	64	16
8	3	8	64	512	256
16	4	16	256	4096	65,536
32	5	32	1,024	32,768	
64	6	64	4,096	262,144	4,294,967,296
128	7	128	16,384	2,097,152	3.4×10^{38}

n	$\log_2 n$	$n \log_2 n$	n^2	n^3
10	3.3	33	100	1,000
50	5.6	280	2,500	125,000
100	6.6	660	10,000	1,000,000
500	8.9	4,450	250,000	125,000,000
1,000	9.9	9,900	1,000,000	1,000,000,000

→ In other words, an algorithm that is $\Theta(n \log n)$ is more efficient than an algorithm that is $\Theta(n^2)$.

Growth Rate Classes



Growth Class	Meaning
O(1)	Constant
O(log n)	Logarithmic
O(n)	Linear
O(n log n)	Log-linearithmic
O(n²)	Quadratic
O(n³)	Cubic (Polynomial)
O(2ⁿ)	Exponential

Key Points :

- Shows how the running time grows as the input size (n) increases.
- Lower growth rate is more efficient.
- Linear time (O(n)) grows faster than logarithmic (O(log n)) but slower than log-linearithmic (O(n log n)).
- Focus is on the growth rate for large n. (ignoring constant factors).

Now You Analyze....

1. Loops

```
for (int i = 0; i < n; i++) {
    ...
}
```

The loop runs n times.
Each iteration takes constant time.

T(n) = O(n)

2. Nested loops

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        ...
    }
}
```

The outer loop runs n times and for each iteration the inner loop runs n times.
Total iterations = n × n.

T(n) = $O(n^2)$

3. Consecutive statements

```
for (int i = 0; i < n; i++) {
    ...
}
for (int j = 0; j < m; j++) {
    ...
}
```

The first loop runs n times and the second loop runs m times. They are independent, so we add their complexities.

T(n) = O(n + m)

4. If-then-else statement

```
if (n > 1000) {
    for (int i = 0; i < n; i++) {
        ...
    }
} else {
    for (int j = 0; j < log(n); j++) {
        ...
    }
}
```

Only one of the two branches is executed.
The first branch runs n times and the second branch runs log(n) times.
We consider the more expensive branch (worst case).

T(n) = O(n)

5. Logarithmic complexity

```
while (n > 1) {
    n = n / 2;
    ...
}
```

In each iteration, n is divided by 2, so the value of n becomes n/2, n/4, n/8, ...
The loop stops when $n \leq 1$.
The number of iterations grows logarithmically with n.

T(n) = O(log n)

Barometer Method

1 Given Algorithm

```
for (int i = 0; i < n; i++) {
    sum = sum + i;
}
```

2 Choose a barometer operation

Identify the operation that is repeated most and contributes most to the running time.

```
sum = sum + i;
```

↑ Barometer operation (addition)

3 Count the executions

The loop runs for $i = 0, 1, 2, \ldots, n - 1$ which is n times.

The barometer operation executes once in each iteration.

Executes: n times

4 Time Complexity

Total running time is proportional to the number of executions of the barometer operation, which is n.

$$T(n) = n = O(n)$$

Key Idea

The barometer method analyzes time complexity by counting how many times the most significant (repeated) operation executes as the input size (n) grows.

Key Takeaways

1. **Algorithm**
   - A step-by-step procedure to solve a problem.
   - We analyze it to find how the running time grows with the input size (n).
2. **Time Complexity**
   - Measures the amount of time an algorithm takes as a function of the input size (n).
   - We express it using Big-O notation.
3. **Barometer Method**
   - Choose the operation that repeats most and contributes most to the running time.
   - Count how many times that operation executes as n grows.
4. **Loops**
   - A single loop that runs n times has time complexity $O(n)$.
   - Each iteration takes constant time.
5. **Nested Loops**
   - Multiply the number of iterations of each loop.
   - Two nested loops each running n times give $O(n^2)$.
6. **Consecutive Statements / Loops**
   - If multiple independent parts execute one after another, add their complexities.
   - Example: $O(n)$ followed by $O(m)$ gives $O(n + m)$.
7. **If-then-else Statement**
   - Only one of the branches is executed.
   - Consider the more expensive branch (worst case) for the complexity.
8. **Logarithmic Complexity**
   - If the input size is divided by a constant (e.g., 2) in each iteration, the number of iterations is logarithmic.
   - This gives $O(\log n)$.